DESIGN AND ANALYSIS OF ALGORITHM

LAB EXPERIMENT -8

NAME:
KOLATHUR SURYA
ROLL NO:
CH.SC.U4CSE24224

1. HUFFMAN CODING

CODE:

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

struct Node{
    char data;
    unsigned freq;
    struct Node *left,*right;
};

struct Node* newNode(char data,unsigned freq){
    struct Node* temp=(struct Node*)malloc(sizeof(struct Node));
    temp->left=temp->right=NULL;
    temp->data=data;
    temp->freq=freq;
    return temp;
}

struct MinHeap{
    unsigned size;
    unsigned capacity;
    struct Node** array;
};

struct MinHeap* createMinHeap(unsigned capacity){
    struct MinHeap* minHeap=(struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size=0;
    minHeap->capacity=capacity;
    minHeap->array=(struct Node**)malloc(capacity*sizeof(struct Node*));
    return minHeap;
}

void swapNode(struct Node** a,struct Node** b){
    struct Node* t=*a;
    *a=*b;
    *b=t;
}

void minHeapify(struct MinHeap* minHeap,int idx){
    int smallest=idx;
    int left=2*idx+1;
    int right=2*idx+2;

    if(left<minHeap->size && minHeap->array[left]->freq < minHeap->array[smallest]->freq)
        smallest=left;

    if(right<minHeap->size && minHeap->array[right]->freq < minHeap->array[smallest]->freq)
        smallest=right;

    if(smallest!=idx){
        swapNode(&minHeap->array[smallest],&minHeap->array[idx]);
        minHeapify(minHeap,smallest);
    }
```

```
    }
}

int isSizeOne(struct MinHeap* minHeap){
    return (minHeap->size==1);
}

struct Node* extractMin(struct MinHeap* minHeap){
    struct Node* temp=minHeap->array[0];
    minHeap->array[0]=minHeap->array[minHeap->size-1];
    minHeap->size--;
    minHeapify(minHeap,0);
    return temp;
}

void insertMinHeap(struct MinHeap* minHeap,struct Node* node){
    minHeap->size++;
    int i=minHeap->size-1;

    while(i && node->freq < minHeap->array[(i-1)/2]->freq){
        minHeap->array[i]=minHeap->array[(i-1)/2];
        i=(i-1)/2;
    }

    minHeap->array[i]=node;
}

struct Node* buildHuffmanTree(char data[],int freq[],int size){
    struct Node *left,*right,*top;

    struct MinHeap* minHeap=createMinHeap(size);

    for(int i=0;i<size;i++)
        minHeap->array[i]=newNode(data[i],freq[i]);

    minHeap->size=size;

    for(int i=(minHeap->size-1)/2;i>=0;i--)
        minHeapify(minHeap,i);

    while(!isSizeOne(minHeap)){
        left=extractMin(minHeap);
        right=extractMin(minHeap);

        top=newNode('$',left->freq+right->freq);
        top->left=left;
        top->right=right;

        insertMinHeap(minHeap,top);
    }
```

```
    return extractMin(minHeap);
}

void printCodes(struct Node* root,int arr[],int top){
    if(root->left){
        arr[top]=0;
        printCodes(root->left,arr,top+1);
    }

    if(root->right){
        arr[top]=1;
        printCodes(root->right,arr,top+1);
    }

    if(!root->left && !root->right){
        printf("%c: ",root->data);
        for(int i=0;i<top;i++)
            printf("%d",arr[i]);
        printf("\n");
    }
}

int main(){

    printf("Name : Kolathur Surya\n");
    printf("Roll No : CH.SC.U4CSE24224\n\n");

    char arr[]={'A','B','C','D','E','F'};
    int freq[]={5,9,12,13,16,45};

    int size=6;

    struct Node* root=buildHuffmanTree(arr,freq,size);

    int code[100],top=0;

    printf("Huffman Codes:\n");
    printCodes(root,code,top);

    return 0;
}
```

OUTPUT:

```
surya@DESKTOP-FJ2EKA4:~$ nano huffman.c
surya@DESKTOP-FJ2EKA4:~$  gcc huffman.c -o huffman
surya@DESKTOP-FJ2EKA4:~$ ./huffman
Name : Kolathur Surya
Roll No : CH.SC.U4CSE24224

Huffman Codes:
F: 0
C: 100
D: 101
A: 1100
B: 1101
E: 111
surya@DESKTOP-FJ2EKA4:~$
```

INTERPRETATION:

Huffman coding is a greedy algorithm used for data compression by assigning shorter binary codes to characters with higher frequencies. It builds a binary tree by repeatedly combining the two nodes with the smallest frequencies until a single tree is formed.